

Introduction à Python intermédiaire

Python intermédiaire orienté traitements métier

- Public : profils métier ayant déjà suivi le cours débutant
- Objectif : automatiser, contrôler et fiabiliser des traitements de données
- Angle : notebook pour explorer, script .py pour produire

Cadrage

- Formation de 3 jours
- Niveau : intermédiaire
- Périmètre : Polars, notebook, Excel, requests, logging, debugger
- Hors périmètre : sujets avancés non nécessaires à ce parcours

Public et prérequis

- Avoir déjà manipulé variables, conditions, boucles et fonctions simples
- Savoir exécuter un script Python dans VS Code ou un terminal
- Comprendre le principe d'un fichier CSV
- Ne pas être développeur n'est pas un problème

Modalité pédagogique

- 30% théorie / 70% pratique
- Beaucoup de démonstration sur cas métier
- Un fil rouge unique sur les 3 jours
- Quiz flash et corrections collectives

Objectifs de sortie

À la fin du cours, les participants savent :

- lire un CSV et un Excel avec Polars
- créer des colonnes dérivées sans détruire la donnée source
- utiliser un notebook pour comprendre une transformation
- transformer cette logique en script .py
- appeler une API simple et joindre les données

Stack du cours

- Python 3.12+
- VS Code
- extension Python
- extension Jupyter
- polars
- altair
- requests

Règles pédagogiques

- `import polars as pl`
- notebook = exploration et visualisation
- script .py = livrable principal
- colonnes sources conservées
- transformations via colonnes dérivées

Organisation des livrables

- `analyse_metier.ipynb`
- `script_final.py`
- `output/resultat_final.csv` ou `output/resultat_final.xlsx`
- `output/anomalies.csv`
- `README_execution.md`

Fil rouge métier

Cas fil rouge sur les 3 jours :

- un export brut à nettoyer
- un notebook pour explorer et contrôler
- un script pour rejouer le traitement
- une API pour enrichir les données
- un export final propre + anomalies

Plan des 3 jours

- Jour 1 : notebook, Polars, premières transformations
- Jour 2 : contrôle qualité, Excel, passage au script, debugger
- Jour 3 : API, configuration, logging, mini-projet final

Jour 1

**Notebook + Polars + premières
transformations**

Jour 1 - objectifs

- faire la transition depuis le niveau débutant
- comprendre le rôle du notebook
- découvrir Polars sur un cas utile
- construire un premier pipeline simple

Ce qu'on reprend du niveau débutant

- lecture de code simple
- variables et fonctions
- structures conditionnelles
- manipulation basique de chaînes

Comment lire un petit script de transformation

```
rows = lire_lignes("input/clients.csv")
rows_ok = [row for row in rows if row["statut"] == "ok"]

for row in rows_ok:
    row["email_clean"] = row["email"].lower()

ecrire_lignes("output/clients_ok.csv", rows_ok)
```

- entrée : input/clients.csv
- filtre : on garde seulement les lignes avec statut == "ok"
- transformation : on crée email_clean
- sortie : output/clients_ok.csv

Dans quel ordre lire ce script

1. repérer le fichier d'entrée
 2. repérer la condition de filtrage
 3. repérer la colonne créée ou modifiée
 4. repérer le fichier de sortie
- inutile de comprendre chaque détail au premier passage
 - au début, il faut surtout comprendre le trajet de la donnée

Boucle simple vs map / filter

```
emails = [" A@x.com ", " ", "B@Y.COM"]
```

```
emails_clean = []  
for email in emails:  
    if email.strip():  
        emails_clean.append(email.strip().lower())
```

- une boucle for est souvent la forme la plus lisible
- map transforme
- filter sélectionne
- il faut savoir les lire, pas en abuser

map

```
nombres = [1, 2, 3]  
resultat = list(map(lambda x: x * 2, nombres))
```

- résultat : [2, 4, 6]
- utile pour transformer une liste simple

filter

```
nombres = [1, 2, 3, 4]
```

```
resultat = list(filter(lambda x: x % 2 == 0, nombres))
```

- résultat : [2, 4]
- utile pour garder seulement certains éléments

map / filter : quand c'est utile, quand éviter

- utile sur listes courtes et cas simples
- utile pour lire du code existant
- à éviter si la lisibilité baisse
- pour les tableaux métier, Polars sera souvent plus clair

Transition : de map / filter vers notebook puis Polars

- sur une liste, on transforme élément par élément
- le notebook sert à tester cette logique pas à pas
- sur un tableau métier, on raisonne ensuite en colonnes et en lignes
- Polars arrive quand la structure et le volume augmentent

Exercice J1-A - Réécrire une boucle avec filter puis map

Consigne :

- partir de cette liste :

```
emails = [" A@x.com ", " ", "B@Y.COM", " "]
```

- supprimer les valeurs vides après strip()
- mettre les emails restants en minuscules
- produire emails_clean

Correction J1-A

```
emails = [" A@x.com ", " ", "B@Y.COM", "  "]

emails_non_vides = list(filter(lambda email: email.strip(), emails))
emails_clean = list(map(lambda email: email.strip().lower(), emails_non_vides))
```

emails_clean

```
["a@x.com", "b@y.com"]
```

- `filter` enlève les valeurs vides
- `map` transforme les valeurs restantes
- le résultat est correct
- on verra juste après pourquoi ce style devient moins pratique sur un vrai tableau métier

Notebook vs script

```
# notebook
```

```
montants = [1200.5, 980.0, 450.0]
```

```
sum(montants)
```

```
# script
```

```
def main() -> None:
```

```
    montants = [1200.5, 980.0, 450.0]
```

```
    print(sum(montants))
```

- notebook : observer, tester, visualiser
- script : exécuter de bout en bout
- notebook : plus souple
- script : plus stable et plus réutilisable

Notebook dans VS Code

```
x = 10  
y = 20  
x + y
```

- créer un fichier .ipynb
- choisir l'interpréteur Python
- exécuter cellule par cellule
- relancer une cellule en gardant le contexte

Structure d'un notebook

```
# Cellule 1
```

```
fichier = "input/clients.csv"
```

```
# Cellule 2
```

```
colonnes_attendues = ["client", "email", "montant", "statut"]
```

```
# Cellule 3
```

```
len(colonnes_attendues)
```

- cellule markdown : expliquer
- cellule code : transformer
- cellule résultat : observer
- on travaille par étapes courtes

Ce qu'on met dans un notebook

- lecture de données
- aperçu du schéma
- vérifications rapides
- graphiques simples
- comparaison avant / après

Ce qu'on ne met pas dans un notebook

- toute la logique de production finale
- les secrets
- des copier-coller inutiles
- des cellules dans le désordre

Pourquoi Polars dans ce cours

- on manipule des tableaux de données
- on veut lire, filtrer, transformer et exporter rapidement
- Polars est la librairie tabulaire du cours intermédiaire
- on l'utilise de façon simple et lisible

DataFrame : notion clé

Un DataFrame, c'est :

- un tableau structuré
- avec des colonnes nommées
- des types de données
- des opérations de filtrage et de transformation

Exemple de colonnes :

- client
- email
- montant
- statut

Construire un petit DataFrame

```
import polars as pl

df_demo = pl.DataFrame(
    {
        "client": ["Alpha", "Beta"],
        "montant": [1200.50, 980.00],
        "statut": ["ok", "pending"],
    }
)
```

df_demo

- utile pour comprendre l'objet manipulé
- même logique qu'un tableau avec colonnes nommées

import polars as pl

```
import polars as pl
```

- convention unique du cours
- pl sera le préfixe utilisé partout dans les exemples

Lire un CSV avec Polars

```
import polars as pl
```

```
df = pl.read_csv("input/clients.csv")
```

- df est un DataFrame
- c'est notre point de départ pour l'analyse

Aperçu du DataFrame

`df.head()`

- voir les premières lignes
- comprendre la structure générale
- vérifier si les colonnes attendues sont là

Ce que montre head()

shape: (5, 4)

client	email	montant	statut
---	---	---	---
str	str	str	str

- shape = nombre de lignes et colonnes affichées
- les noms de colonnes sont visibles
- les types sont visibles

Schéma, types, nulls

```
df.schema
```

```
df.null_count()
```

- voir les types
- repérer les colonnes problématiques
- compter rapidement les valeurs manquantes

Résumer rapidement les colonnes avec `df.describe()`

`df.describe()`

- utile pour un premier diagnostic rapide
- particulièrement utile sur les colonnes numériques
- permet de voir min, max, moyenne et nombre de valeurs

Expression Polars : `pl.col(...)`

```
pl.col("email")
```

```
pl.col("montant")
```

```
pl.col("email").str.lowercase()
```

- une expression désigne une colonne ou une transformation
- seule, elle ne produit pas encore de résultat visible
- elle s'exécute dans un contexte comme `select`, `with_columns` ou `filter`

Les 3 contextes Polars à connaître

- `select` : choisir ou calculer des colonnes
- `with_columns` : ajouter des colonnes au DataFrame
- `filter` : garder seulement certaines lignes

select

```
df.select(  
    pl.col("client"),  
    pl.col("email"),  
    pl.col("statut"),  
)
```

- sélectionner seulement les colonnes utiles
- alléger l'analyse

filter avec Polars

```
df.filter(pl.col("statut") == "ok")
```

- garder uniquement les lignes utiles
- base du contrôle qualité

sort

```
df.sort("montant", descending=True)
```

- prioriser l'analyse
- faire émerger les cas extrêmes

Exercice J1-B - Explorer un export brut

Consigne :

- charger un CSV
- afficher les 5 premières lignes
- afficher le schéma
- repérer les colonnes à surveiller

Correction J1-B

```
import polars as pl

df = pl.read_csv("input/clients.csv")

print(df.head())
print(df.schema)
print(df.null_count())
print(df.describe())
```

- lecture du fichier
- aperçu rapide
- schéma vérifié
- colonnes avec valeurs manquantes repérées
- résumé statistique affiché

Créer des colonnes avec `with_columns`

```
df = df.with_columns(  
    pl.col("email").str.to_lowercase().alias("email_clean")  
)
```

- le cœur de la transformation métier
- `with_columns` ajoute des colonnes
- contrairement à `select`, on garde aussi les colonnes déjà présentes

Nettoyage texte

```
df = df.with_columns(  
    pl.col("email")  
        .str.strip_chars()  
        .str.to_lowercase()  
        .alias("email_clean")  
)
```

- strip_chars()
- to_lowercase()
- replace()
- normaliser avant de contrôler

Le pont avec `map` Python :

`map_elements()`

```
def clean_email(value: str) -> str:  
    return value.strip().lower()
```

```
df = df.with_columns(  
    pl.col("email")  
        .map_elements(clean_email, return_dtype=pl.String)  
        .alias("email_clean")  
)
```

- `map_elements()` applique une fonction Python valeur par valeur
- c'est le pont naturel avec `map(...)` vu plus tôt
- toujours préciser `return_dtype`

Préférer les expressions natives quand c'est possible

```
df = df.with_columns(  
    pl.col("email")  
        .str.strip_chars()  
        .str.to_lowercase()  
        .alias("email_clean")  
)
```

- ici, l'expression native est meilleure que `map_elements()`
- plus lisible
- plus rapide
- `map_elements()` sert surtout quand la logique n'existe pas déjà dans Polars

Normaliser un montant

```
df = df.with_columns(  
    pl.col("montant")  
        .str.replace_all("€", "")  
        .str.replace_all(" ", "")  
        .str.replace(",", ".")  
        .cast(pl.Float64, strict=False)  
        .alias("montant_net")  
)
```

- supprimer les espaces
- gérer , et .
- convertir en nombre
- ranger le résultat dans une nouvelle colonne

Calculer TVA et TTC avec des expressions Polars

```
TAUX_TVA = 0.20
```

```
df = df.with_columns([
    (pl.col("montant_net") * TAUX_TVA).round(2).alias("montant_tva"),
    (pl.col("montant_net") * (1 + TAUX_TVA)).round(2).alias("montant_ttc"),
])
```

- dans cet exemple, montant_net est notre base HT
- `pl.col("montant_net") * TAUX_TVA` calcule la TVA
- `pl.col("montant_net") * (1 + TAUX_TVA)` calcule le TTC
- on ajoute des colonnes métier sans écraser la source

Si la source est TTC, retrouver le HT

```
TAUX_TVA = 0.20
```

```
df = df.with_columns([
    (pl.col("montant_ttc") / (1 + TAUX_TVA)).round(2).alias("montant_ht"),
    (
        pl.col("montant_ttc")
        - (pl.col("montant_ttc") / (1 + TAUX_TVA))
    ).round(2).alias("montant_tva"),
])
```

- ici la colonne de départ est `montant_ttc`
- on reconstruit le HT
- on en déduit la TVA
- même logique : on ajoute des colonnes calculées

Exercice express - Calculer TVA et TTC

Consigne :

- partir d'une colonne montant_net considérée comme HT
- ajouter montant_tva
- ajouter montant_ttc
- utiliser un taux de 20%

Correction - Calculer TVA et TTC

```
TAUX_TVA = 0.20
```

```
df = df.with_columns([  
    (pl.col("montant_net") * TAUX_TVA).round(2).alias("montant_tva"),  
    (pl.col("montant_net") * (1 + TAUX_TVA)).round(2).alias("montant_ttc"),  
])
```

- une expression par colonne calculée
- résultat arrondi à 2 décimales
- aucun écrasement de la colonne source

Normaliser une date

```
df = df.with_columns(  
    pl.col("date_commande")  
        .str.strptime(pl.Date, format="%d/%m/%Y", strict=False)  
        .dt.strftime("%Y-%m-%d")  
        .alias("date_commande_clean")  
)
```

- identifier le format source
- convertir dans un format stable
- garder la colonne source si utile
- créer une colonne date propre

Normaliser un statut

```
df = df.with_columns(  
    pl.col("statut")  
        .str.strip_chars()  
        .str.to_lowercase()  
        .replace({"pending": "en_attente", "ok": "valide"})  
        .alias("statut_normalise")  
)
```

- mettre en minuscules
- supprimer les espaces parasites
- ramener plusieurs variantes vers une même valeur

Règle non négociable : colonnes dérivées

- on garde les colonnes sources
- on ajoute `email_clean`, `montant_net`, etc.
- on évite l'écrasement destructif
- on rend la transformation traçable

Assembler plusieurs colonnes dérivées dans un même pipeline

```
df = df.with_columns([
    pl.col("email").str.strip_chars().str.to_lowercase().alias("email_clean"),
    pl.col("montant").str.replace(",", ".").cast(pl.Float64, strict=False).alias("montant_net"),
    pl.col("statut").str.strip_chars().str.to_lowercase().alias("statut_normalise"),
])
```

- email_clean : email nettoyé
- montant_net : montant converti
- statut_normalise : statut harmonisé
- un seul with_columns peut porter plusieurs transformations lisibles

Exercice J1-C - Ajouter une colonne de contrôle au pipeline

Consigne :

- repartir du pipeline précédent
- ajouter `ligne_valide`
- `ligne_valide` vaut `True` si `email_clean` contient `@`
- `ligne_valide` vaut `True` si `montant_net` n'est pas nul
- garder les colonnes sources

Correction J1-C

```
df = df.with_columns([
    pl.col("email").str.strip_chars().str.to_lowercase().alias("email_clean"),
    pl.col("montant").str.replace(",", ".").cast(pl.Float64, strict=False).alias("montant_net"),
    pl.col("statut").str.strip_chars().str.to_lowercase().alias("statut_normalise"),
]).with_columns(
    (
        pl.col("email_clean").str.contains("@", literal=True)
        & pl.col("montant_net").is_not_null()
    ).alias("ligne_valide")
)
```

- transformation via with_columns
- conservation de la source
- logique de validation séparée
- sortie lisible

Atelier Jour 1 - Nettoyer un export métier simple

Objectif :

- lire un CSV
- créer les colonnes dérivées attendues
- produire une première table exploitable

Squelette atelier J1

- lecture du fichier
- analyse rapide
- ajout des colonnes
- vérification sur quelques lignes
- export d'une sortie intermédiaire

Livrable Jour 1

- notebook d'exploration propre
- première transformation exécutable
- sortie intermédiaire compréhensible

Quiz flash Jour 1

- rôle du notebook
- rôle du script
- utilité de `with_columns`
- calculer une colonne TVA ou TTC avec une expression
- retrouver un montant HT à partir d'un TTC
- pourquoi garder les colonnes sources

Récapitulatif Jour 1

- lire
- comprendre
- filtrer
- créer des colonnes dérivées
- faire un calcul métier simple avec des expressions Polars
- manipuler un cas HT / TVA / TTC
- préparer un vrai pipeline

Jour 2

**Contrôle qualité + Excel +
passage au script**

Jour 2 - objectifs

- mesurer la qualité des données
- visualiser les résultats
- intégrer Excel
- passer du notebook au script
- apprendre à déboguer

Rappel du fil rouge

- export brut
- nettoyage
- contrôle qualité
- export final
- future intégration API

Pourquoi faire du contrôle qualité

- vérifier qu'on ne produit pas de faux résultats
- repérer les anomalies métier
- expliquer la transformation
- sécuriser le livrable final

Agréger = résumer un tableau

```
resume = df.select([
    pl.len().alias("nb_lignes"),
    pl.col("montant_net").sum().alias("montant_total"),
    pl.col("montant_net").mean().alias("montant_moyen"),
])
```

- on passe de milliers de lignes à quelques indicateurs
- on commence souvent par ce niveau de synthèse

Regrouper pour comparer des statuts

```
df.groupby("statut_normalise").agg([  
    pl.len().alias("nb_lignes"),  
    pl.col("montant_net").sum().alias("montant_total"),  
])
```

- base des synthèses métier

Exemple de sortie agrégée

statut_normalise	nb_lignes	montant_total
valide	120	185430.50
en_attente	34	24100.00

- une ligne par groupe
- pratique pour les contrôles et les graphiques

Quels indicateurs regarder d'abord

```
df.select([
    pl.len().alias("nb_lignes"),
    pl.col("montant_net").sum().alias("montant_total"),
    pl.col("montant_net").mean().alias("montant_moyen"),
])
```

- len : compter les lignes
- sum : sommer un indicateur
- mean : moyenne simple
- indispensables pour un mini diagnostic qualité

Vérifier qu'un client n'apparaît pas plusieurs fois

```
df.groupby("email_clean").agg(  
    pl.len().alias("nb_occurrences")  
).filter(pl.col("nb_occurrences") > 1)
```

- identifier une clé logique
- compter les répétitions
- distinguer doublon technique et métier

Mesurer ce qui bloque l'exploitation

```
df.select([
    pl.col("ligne_valide").not_().sum().alias("nb_invalides"),
    pl.col("email_clean").is_null().sum().alias("nb_emails_vides"),
    pl.col("montant_net").is_null().sum().alias("nb_montants_invalides"),
])
```

- lignes invalides
- dates absentes
- emails incorrects
- montants non exploitables

Vérifier l'effet réel du nettoyage

```
df_clean = df.filter(pl.col("ligne_valide"))

avant = df.select(pl.len().alias("nb_lignes"))
apres = df_clean.select([
    pl.len().alias("nb_lignes"),
    pl.col("ligne_valide").sum().alias("nb_valides"),
])
```

- nombre de lignes
- nombre d'anomalies
- nombre de lignes valides
- évolution des statuts

Exercice J2-A - Construire un mini diagnostic qualité

Consigne :

- compter les lignes
- compter les anomalies
- compter les lignes valides
- produire une synthèse courte

Correction J2-A

```
diagnostic = df.groupby("statut_normalise").agg([
    pl.len().alias("nb_lignes"),
    pl.col("ligne_valide").sum().alias("nb_valides"),
    pl.col("montant_net").sum().alias("montant_total"),
]).sort("nb_lignes", descending=True)
```

- group_by
- agg
- indicateurs simples
- sortie compréhensible pour un métier

Pourquoi un graphique ici

- rendre un résultat lisible immédiatement
- aider à expliquer la qualité des données
- voir vite les déséquilibres

Passer d'un diagnostic à un graphique

```
df_chart = df.groupby("statut_normalise").agg(  
    pl.len().alias("nb_lignes")  
)
```

- un graphique part d'une table déjà agrégée
- Altair n'est pas là pour nettoyer les données
- le nettoyage et les comptages se font d'abord avec Polars

Graphique 1 - Répartition des statuts

```
import altair as alt
```

```
alt.Chart(df_chart).mark_bar().encode(  
    x=alt.X("statut_normalise:N", title="Statut"),  
    y=alt.Y("nb_lignes:Q", title="Nombre de lignes"),  
)
```

- nombre de lignes par statut
- lecture rapide d'un volume par catégorie

Préparer la table des anomalies

```
df_anomalies = pl.DataFrame(  
    {  
        "type_anomalie": ["email_invalide", "montant_invalide", "date_invalide"],  
        "nb_lignes": [12, 5, 3],  
    }  
)
```

- un graphique a besoin d'une table simple
- ici : une catégorie + un volume

Graphique 2 - Nombre d'anomalies

```
alt.Chart(df_anomalies).mark_bar(color="crimson").encode(  
    x="type_anomalie:N",  
    y="nb_lignes:Q",  
)
```

- anomalies par type
- visualisation des problèmes prioritaires

Préparer la volumétrie par catégorie

```
df_categories = df.groupby("categorie").agg(  
    pl.col("montant_net").sum().alias("montant_total")  
)
```

- table d'entrée simple
- une catégorie, un total

Graphique 3 - Volumétrie par catégorie

```
alt.Chart(df_categories).mark_bar().encode(  
    x="categorie:N",  
    y="montant_total:Q",  
)
```

- catégorie métier
- volume ou montant associé
- aide à prioriser les contrôles

Interpréter un graphique métier

- regarder les volumes
- regarder les écarts
- ne pas surinterpréter
- relier toujours au fichier source

Exercice J2-B - Produire 2 graphiques utiles

Consigne :

- un graphique de statuts
- un graphique d'anomalies ou de volumétrie
- expliquer en une phrase ce qu'ils montrent

Correction J2-B

```
chart = alt.Chart(df_chart).mark_bar().encode(  
    x="statut_normalise:N",  
    y="nb_lignes:Q",  
    tooltip=["statut_normalise", "nb_lignes"],  
).properties(title="Répartition des statuts")
```

- graphique lisible
- titre clair
- axes compréhensibles
- utilité métier explicite

Quand le fichier d'entrée est un Excel

```
df = pl.read_excel("input/entree.xlsx")
```

- utile quand le métier travaille d'abord sous Excel

Ce qu'il faut installer pour lire Excel

- nécessaire pour la lecture Excel dans ce cours
- à installer avant la session
- à tester sur les postes avant le jour J

Quand le livrable final doit être un Excel

```
df.write_excel("output/resultat.xlsx")
```

- sortie métier fréquente

Ce qu'il faut installer pour écrire Excel

- nécessaire pour l'écriture Excel
- à intégrer dans l'environnement standard du cours

Quand sortir en CSV, quand sortir en Excel

- CSV : traitement, échange simple, robustesse
- Excel : restitution métier, lecture humaine, diffusion

Exercice J2-C - Lire un Excel et réécrire une sortie propre

Consigne :

- lire un fichier Excel
- appliquer une transformation simple
- réécrire une sortie propre

Correction J2-C

```
import polars as pl

df = pl.read_excel("input/entree.xlsx")

result = df.with_columns(
    pl.col("email").str.strip_chars().str.to_lowercase().alias("email_clean")
)

result.write_excel("output/resultat.xlsx")
```

- lecture Excel
- transformation simple et lisible
- export rejouable vers Excel

Quand quitter le notebook

```
# notebook : exploration
df = pl.read_csv("input/clients.csv")
df.head()

# script : production
output_df = transform_clients(df)
output_df.write_csv("output/resultat.csv")
```

- quand la logique devient stable
- quand il faut rejouer
- quand il faut partager un livrable
- quand il faut industrialiser

Stabiliser le traitement dans un vrai script

```
def load_input(path: str) -> pl.DataFrame:  
    return pl.read_csv(path)
```

```
def transform_clients(df: pl.DataFrame) -> pl.DataFrame:  
    return df.with_columns(...)
```

- une fonction = une responsabilité
- améliorer la lisibilité
- faciliter le test manuel

Script minimal rejouable de bout en bout

```
def load_input(path: str) -> pl.DataFrame:
    return pl.read_csv(path)

def transform_clients(df: pl.DataFrame) -> pl.DataFrame:
    return df.with_columns(...)

def main() -> None:
    df = load_input("input/clients.csv")
    result = transform_clients(df)
    result.write_csv("output/resultat.csv")

if __name__ == "__main__":
    main()
```

- centraliser l'enchaînement
- séparer chargement, transformation et export
- rendre le script exécutable directement

Arborescence input / output / secret / libs

```
projet/  
  input/  
  output/  
  secret/  
  libs/  
  main.py
```

- input/ : données d'exemple
- output/ : résultats
- secret/ : hors contexte
- libs/ : utilitaires génériques

Conserver libs générique et métier dans main.py

- `libs/` pour ce qui se réutilise
- `main.py` pour la logique locale du script
- ne pas tout mettre dans `libs/`

Exemple : transformer une cellule notebook en fonction

```
# notebook
df = df.with_columns(
    pl.col("email").str.to_lowercase().alias("email_clean")
)

# script
def add_clean_email(df: pl.DataFrame) -> pl.DataFrame:
    return df.with_columns(
        pl.col("email").str.to_lowercase().alias("email_clean")
    )
```

- prendre un bout de code stable
- l'extraire dans une fonction nommée
- rappeler l'intérêt : rejouer, lire, corriger

Exercice J2-D - Transformer une logique notebook en script

Consigne :

- prendre une transformation validée en notebook
- la mettre dans une fonction
- l'appeler depuis `main()`

Correction J2-D

```
def transform_clients(df: pl.DataFrame) -> pl.DataFrame:
    return df.with_columns(
        pl.col("email").str.strip_chars().str.to_lowercase().alias("email_clean")
    )

def main() -> None:
    df = pl.read_csv("input/clients.csv")
    transform_clients(df).write_csv("output/resultat.csv")

if __name__ == "__main__":
    main()
```

- fonction claire
- point d'entrée propre
- lecture du fichier
- export du résultat

Quand le résultat n'est pas celui attendu

- un script ne fait pas toujours ce qu'on croit
- lire le code ne suffit pas toujours
- le debugger évite de travailler à l'aveugle

Arrêter l'exécution au bon moment

```
def normalize_email(email: str) -> str:  
    return email.strip().lower()
```

```
email = normalize_email(" CONTACT@X.FR ")  
print(email)
```

- arrêter l'exécution à un point précis
- regarder l'état intermédiaire

Avancer sans perdre le fil

- avancer ligne par ligne
- entrer dans une fonction si nécessaire
- comprendre où la logique dévie

Vérifier la donnée au moment du bug

```
current_email = row["email"]  
email_clean = current_email.strip().lower()
```

- lire la valeur courante
- vérifier si le problème vient de l'entrée ou de la logique

Lire une erreur avant de corriger

FileNotFoundError: [Errno 2] No such file or directory:
'input/clients.csv'

- identifier le message
- repérer la ligne
- comprendre si le problème vient du code
- comprendre si le problème vient de la donnée
- comprendre si le problème vient du chemin de fichier

Exercice J2-E - Déboguer un script qui sort un mauvais résultat

Consigne :

- poser un breakpoint
- inspecter une variable
- identifier la ligne fautive
- expliquer la correction

Correction J2-E

```
# bug observe au debugger
def normalize_email(email: str) -> str:
    return email.lower()
```

```
# correction
def normalize_email(email: str) -> str:
    return email.strip().lower()
```

- breakpoint posé avant la normalisation
- valeur brute inspectée : espaces parasites visibles
- cause du bug identifiée
- correction locale et explicite

Atelier Jour 2 - Sortir un fichier propre + un fichier anomalies

Objectif :

- produire un résultat métier propre
- séparer les anomalies
- disposer d'un script exécutable

Squelette atelier J2

- lecture du fichier
- transformation
- diagnostic qualité
- export résultat
- export anomalies

Livrable Jour 2

- notebook de contrôle
- script structuré
- sortie propre
- fichier d'anomalies

Quiz flash Jour 2

- rôle de group_by
- rôle d'Altair
- différence notebook / script
- rôle du debugger

Récapitulatif Jour 2

- contrôler
- visualiser
- lire / écrire Excel
- structurer un script
- déboguer

Jour 3

API + logging + mini-projet final

Jour 3 - objectifs

- consommer une API simple
- joindre les données à l'existant
- configurer proprement un script
- fiabiliser avec logging
- terminer un mini-projet complet

Ce qu'on doit savoir faire à la fin

- exécuter un traitement complet
- lire une sortie
- expliquer les anomalies
- modifier un paramètre
- livrer un résultat propre

Quand le fichier local ne suffit plus

- une API renvoie des données
- ici : usage simple en lecture
- pas de complexité inutile

Appeler une source externe

```
import requests
```

```
url = "https://api.exemple.local/customers"
```

```
r = requests.get(url, timeout=20)
```

- base du bloc API

Éviter qu'un appel externe bloque tout

- éviter qu'un script reste bloqué
- rendre l'exécution plus robuste

Lire la réponse utile de l'API

```
payload = r.json()
```

- structure fréquente côté API

Exemple de payload JSON

```
[  
  {"customer_id": 101, "city": "Paris", "segment": "B2B"},  
  {"customer_id": 102, "city": "Lyon", "segment": "PME"},  
]
```

- une liste de dictionnaires
- forme classique pour créer un DataFrame

Ramener la réponse API dans le même format de travail

```
df_api = pl.DataFrame(payload).select([  
    "customer_id",  
    "city",  
    "segment",  
])
```

- convertir une réponse exploitable en table

Enrichir le fichier local avec la donnée externe

```
df_local = pl.read_csv("input/clients.csv")
```

```
df_final = df_local.join(  
    df_api,  
    on="customer_id",  
    how="left",  
)
```

- enrichir un export existant
- aligner une clé
- produire une table consolidée

Ne pas planter silencieusement sur un appel API

```
try:
    r = requests.get(url, timeout=20)
    r.raise_for_status()
    payload = r.json()
except requests.RequestException as exc:
    print(f"Erreur API: {exc}")
```

- vérifier le code de retour
- utiliser try / except
- garder un message clair

Exercice J3-A - Récupérer et transformer une réponse API

Consigne :

- appeler une API simple
- lire le JSON
- produire une table Polars
- préparer une jointure

Correction J3-A

```
import requests
import polars as pl

r = requests.get(url, timeout=20)
r.raise_for_status()
payload = r.json()
df_api = pl.DataFrame(payload).select([
    "customer_id",
    "city",
    "segment",
])
```

- requête correcte
- timeout présent
- JSON exploité
- table prête pour la suite

Pourquoi ne jamais mettre un token dans le code

- fuite de secret
- partage involontaire
- maintenance difficile

Paramétrer le script sans modifier le code

```
export API_TOKEN="token_exemple"  
python script_final.py
```

- bon endroit pour un token ou une clé
- permet de garder le code propre

Lire la configuration côté Python

```
import os
```

```
token = os.environ.get("API_TOKEN")
```

- simple
- standard
- sécurisé à l'échelle du cours

Séparer données réelles et zone de travail

- stocker ce qui ne doit pas être diffusé
- ne jamais l'utiliser comme support de démo
- travailler sur des fichiers d'exemple

Travailler sur un faux jeu de données fidèle

- même structure
- fausses données
- même logique de traitement

Exercice J3-B - Paramétrer un script avec token et chemins

Consigne :

- lire un token depuis l'environnement
- paramétrer l'entrée et la sortie
- garder le code sans secret en dur

Correction J3-B

```
import os

token = os.environ.get("API_TOKEN")
input_path = os.environ.get("INPUT_PATH", "input/clients.csv")
output_path = os.environ.get("OUTPUT_PATH", "output/resultat.csv")

if not token:
    raise SystemExit("API_TOKEN manquant")
```

- variable d'environnement lue correctement
- chemins séparés
- aucun secret dans le code

Suivre ce que fait le script en production

- suivre ce que fait le script
- comprendre un échec
- garder une trace utile

Mettre en place une trace minimale

```
import logging
```

```
logging.basicConfig(  
    level=logging.INFO,  
    format="%(levelname)s %(message)s",  
)
```

- une seule initialisation au démarrage
- niveau simple et lisible pour le cours

Dire ce que le script est en train de lancer

```
input_path = "input/clients.csv"
```

```
logging.info("Demarrage du traitement clients")  
logging.info("Fichier source: %s", input_path)
```

- nom du traitement
- fichier d'entrée
- horodatage

Rendre le volume traité visible

```
df = pl.read_csv(input_path)
result = transform_clients(df)
```

```
# apres lecture
logging.info("Lignes lues: %s", df.height)
```

```
# apres transformation
logging.info("Lignes exportees: %s", result.height)
```

- nombre de lignes lues
- nombre de lignes sorties
- nombre d'anomalies

Signaler les problèmes sans bloquer

```
nb_invalides = result.filter(pl.col("ligne_valide").not_()).height
```

```
logging.warning("Emails invalides: %s", nb_invalides)
```

- type d'anomalie
- volume concerné
- signal utile sans bruit excessif

Garder une erreur exploitable

```
try:  
    r = requests.get(url, timeout=20)  
    r.raise_for_status()  
except requests.RequestException:  
    logging.exception("Echec pendant l'appel API")
```

- message clair
- contexte minimal
- pas de journal incompréhensible

Encadrer seulement les zones risquées

```
try:
    result = transform_clients(df)
except ValueError as exc:
    logging.error("Transformation invalide: %s", exc)
    raise
```

- entourer les zones risquées
- ne pas masquer toutes les erreurs
- garder un message exploitable

Assembler un traitement rejouable et traçable

```
import logging
import os
import requests
import polars as pl

logging.basicConfig(level=logging.INFO)
token = os.environ.get("API_TOKEN")
url = os.environ.get("API_URL", "https://api.exemple.local/customers")
output_path = os.environ.get("OUTPUT_PATH", "output/resultat.csv")

if not token:
    raise SystemExit("API_TOKEN manquant")

logging.info("Debut du traitement")
df = pl.read_csv("input/clients.csv")
r = requests.get(url, timeout=20)
r.raise_for_status()
df_api = pl.DataFrame(r.json())
result = df.join(df_api, on="customer_id", how="left")
result.write_csv(output_path)
logging.info("Fin du traitement")
```

- lecture paramétrée
- appel API
- jointure
- logs
- export

Exercice J3-C - Ajouter des logs utiles à un script

Consigne :

- ajouter un log de démarrage
- ajouter un log de volumétrie
- ajouter un log d'erreur

Correction J3-C

```
import logging

logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")

logging.info("Debut du traitement")
logging.info("Lignes lues: %s", df.height)

try:
    result = transform_clients(df)
except ValueError:
    logging.exception("Echec de transformation")
    raise

logging.info("Lignes exportees: %s", result.height)
```

- logs courts
- logs lisibles
- logs réellement utiles à l'exploitation

Mini-projet final

- reprendre le fil rouge complet
- de l'entrée brute jusqu'au livrable final

Entrées attendues

- fichier source métier
- paramètres d'exécution
- éventuellement une réponse API simple

Étapes du traitement

- lire
- nettoyer
- contrôler
- enrichir
- exporter

Sorties attendues

- fichier final exploitable
- fichier anomalies
- script lisible
- README d'exécution

Checklist de validation

- les colonnes attendues existent
- la volumétrie est cohérente
- les anomalies sont identifiées
- la sortie est réutilisable

Squelette du projet final

- analyse_metier.ipynb
- script_final.py
- output/resultat_final.csv ou .xlsx
- output/anomalies.csv
- README_execution.md

Atelier final - Notebook + script + export + anomalies

Objectif :

- mobiliser tout le cours
- produire un vrai livrable

Livrable final

- notebook lisible
- script exécutable
- sortie propre
- anomalies séparées
- documentation courte

Critères d'évaluation

- exactitude fonctionnelle
- lisibilité
- robustesse minimale
- cohérence des sorties
- respect de la structure demandée

Modes dégradés

- si l'API est indisponible : réponse locale simulée
- si Excel bloque : sortie CSV temporaire
- si le rythme ralentit : correction guidée sur le mini-projet

Évaluation finale

- sait lire un CSV et un Excel avec Polars
- sait créer des colonnes dérivées
- sait produire un graphique simple
- sait transformer un notebook en script
- sait appeler une API simple
- sait utiliser logging

Fin de formation